

Contents

1	Evaluation Metrics	1
1.1	Comprehensive Performance Assessment Methodology	1
1.2	Table of Contents	2
1.3	1. Overview	2
1.3.1	1.1 Evaluation Dimensions	2
1.4	2. Detection Performance Metrics	2
1.4.1	2.1 Confusion Matrix	2
1.4.2	2.2 Basic Metrics	3
1.4.3	2.3 Advanced Metrics	4
1.4.4	2.4 Multi-Class Metrics	4
1.4.5	2.5 Implementation	5
1.5	3. Efficiency Metrics	6
1.5.1	3.1 Detection Latency	6
1.5.2	3.2 Throughput	7
1.5.3	3.3 Computational Resource Usage	7
1.5.4	3.4 Energy Consumption (for edge devices)	8
1.6	4. Scalability Metrics	9
1.6.1	4.1 Performance vs. Network Size	9
1.6.2	4.2 Distributed System Performance	10
1.7	5. Adaptability Metrics	10
1.7.1	5.1 Zero-Day Attack Detection Rate	10
1.7.2	5.2 Concept Drift Handling	11
1.7.3	5.3 Adaptation Time	12
1.8	6. Explainability Metrics	12
1.8.1	6.1 Feature Importance Consistency	12
1.8.2	6.2 Explanation Fidelity	13
1.8.3	6.3 User Trust Score (Human Evaluation)	14
1.9	7. Privacy Metrics	14
1.9.1	7.1 Differential Privacy Guarantee	14
1.9.2	7.2 Information Leakage	14
1.10	8. Comparative Evaluation Framework	15
1.10.1	8.1 Baseline Comparisons	15
1.10.2	8.2 Comparative Table Template	15
1.11	9. Statistical Significance Testing	15
1.11.1	9.1 Cross-Validation	15
1.11.2	9.2 Statistical Tests	16
1.12	Conclusion	17

1 Evaluation Metrics

1.1 Comprehensive Performance Assessment Methodology

Metrics for Detection Performance, Efficiency, Scalability, Adaptability, and Explainability

Last Updated: December 22, 2025

1.2 Table of Contents

- 1. Overview
 - 2. Detection Performance Metrics
 - 3. Efficiency Metrics
 - 4. Scalability Metrics
 - 5. Adaptability Metrics
 - 6. Explainability Metrics
 - 7. Privacy Metrics
 - 8. Comparative Evaluation Framework
 - 9. Statistical Significance Testing
-

1.3 1. Overview

Comprehensive evaluation of the AI-based IDS requires metrics across multiple dimensions: detection accuracy, computational efficiency, scalability, adaptability to new threats, explainability, and privacy preservation. This document defines all metrics, their mathematical formulations, target values, and measurement methodologies.

1.3.1 1.1 Evaluation Dimensions

Dimension	Key Metrics	Target Goals
Detection Performance	Accuracy, Precision, Recall, F1, AUC-ROC	>98% accuracy, <1% FPR
Efficiency	Latency, Throughput, CPU/Memory usage	<100ms latency, >10K pps
Scalability	Performance vs. size, Load capacity	Linear scaling to 100K devices
Adaptability	Zero-day detection, Drift handling	>95% zero-day detection
Explainability	Feature importance, User trust	High interpretability scores
Privacy	Privacy loss, Information leakage	$\epsilon < 1.0$ (differential privacy)

1.4 2. Detection Performance Metrics

1.4.1 2.1 Confusion Matrix

Predicted

		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Where: - **TP (True Positive)**: Attacks correctly identified as attacks - **TN (True Negative)**: Normal traffic correctly identified as normal - **FP (False Positive)**: Normal traffic incorrectly identified as attacks - **FN (False Negative)**: Attacks incorrectly identified as normal

1.4.2 2.2 Basic Metrics

Accuracy:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

- Measures overall correctness
- Target: **>98%**
- Limitation: Can be misleading with imbalanced datasets

Precision (Positive Predictive Value):

$$\text{Precision} = TP / (TP + FP)$$

- Proportion of predicted attacks that are actual attacks
- Target: **>97%**
- Important for minimizing false alarms

Recall (Sensitivity, True Positive Rate):

$$\text{Recall} = TP / (TP + FN)$$

- Proportion of actual attacks that are detected
- Target: **>98%**
- Critical for security (missing attacks is costly)

Specificity (True Negative Rate):

$$\text{Specificity} = TN / (TN + FP)$$

- Proportion of normal traffic correctly identified
- Target: **>99%**

F1-Score (Harmonic Mean of Precision and Recall):

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

- Balanced measure for imbalanced datasets
- Target: **>97%**

False Positive Rate (FPR):

$$FPR = FP / (FP + TN) = 1 - \text{Specificity}$$

- Proportion of normal traffic incorrectly flagged
- Target: **<1%**
- Critical for operational feasibility

False Negative Rate (FNR, Miss Rate):

$$\text{FNR} = \text{FN} / (\text{FN} + \text{TP}) = 1 - \text{Recall}$$

- Proportion of attacks missed
- Target: <2%

1.4.3 2.3 Advanced Metrics

Matthews Correlation Coefficient (MCC):

$$\text{MCC} = (\text{TP} \times \text{TN} - \text{FP} \times \text{FN}) / \sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}$$

- Range: [-1, 1], where 1 = perfect prediction
- Balanced measure even for imbalanced datasets
- Target: >0.95

Cohen's Kappa:

$$\kappa = (\text{p}_o - \text{p}_e) / (1 - \text{p}_e)$$

where: - p_o : observed agreement (accuracy) - p_e : expected agreement by chance -
Target: >0.90 (almost perfect agreement)

Area Under ROC Curve (AUC-ROC):

ROC curve: Plot of TPR vs. FPR at various thresholds

$$\text{AUC} = \int \text{TPR} \, d(\text{FPR})$$

- Range: [0, 1], where 1 = perfect classifier
- Threshold-independent metric
- Target: >0.99

Area Under Precision-Recall Curve (AUC-PR):

PR curve: Plot of Precision vs. Recall

$$\text{AUC-PR} = \int \text{Precision} \, d(\text{Recall})$$

- Better than AUC-ROC for imbalanced datasets
- Target: >0.98

1.4.4 2.4 Multi-Class Metrics

For multi-class classification (Normal, DoS, Probe, R2L, U2R):

Macro-Average:

$$\text{Macro-Precision} = (1/C) \times \sum \text{Precision}_i$$

$$\text{Macro-Recall} = (1/C) \times \sum \text{Recall}_i$$

$$\text{Macro-F1} = (1/C) \times \sum \text{F1}_i$$

- Equal weight to each class
- Useful when all classes equally important

Weighted-Average:

$\text{Weighted-Precision} = \sum (n_i/N) \times \text{Precision}_i$
 $\text{Weighted-Recall} = \sum (n_i/N) \times \text{Recall}_i$
 $\text{Weighted-F1} = \sum (n_i/N) \times \text{F1}_i$

- Weight by class frequency
- Reflects real-world class distribution

Per-Class Metrics: - Report Precision, Recall, F1 for each attack type - Identify which attacks are harder to detect

1.4.5 2.5 Implementation

```

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             roc_auc_score, roc_curve, matthews_corrcoef,
                             cohen_kappa_score)

import matplotlib.pyplot as plt
import seaborn as sns

def evaluate_detection_performance(y_true, y_pred, y_proba=None):
    """
    Comprehensive detection performance evaluation
    """

    results = {}

    # Basic metrics
    results['accuracy'] = accuracy_score(y_true, y_pred)
    results['precision'] = precision_score(y_true, y_pred, average='weighted')
    results['recall'] = recall_score(y_true, y_pred, average='weighted')
    results['f1_score'] = f1_score(y_true, y_pred, average='weighted')

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    results['confusion_matrix'] = cm

    # FPR, FNR (binary classification)
    if len(np.unique(y_true)) == 2:
        tn, fp, fn, tp = cm.ravel()
        results['fpr'] = fp / (fp + tn)
        results['fnr'] = fn / (fn + tp)
        results['specificity'] = tn / (tn + fp)

    # Advanced metrics
    results['mcc'] = matthews_corrcoef(y_true, y_pred)
    results['kappa'] = cohen_kappa_score(y_true, y_pred)

    # AUC-ROC (if probabilities available)
    if y_proba is not None:

```

```

if len(np.unique(y_true)) == 2:
    results['auc_roc'] = roc_auc_score(y_true, y_proba[:, 1])
else:
    results['auc_roc'] = roc_auc_score(y_true, y_proba,
                                      multi_class='ovr', average='weighted')

# Classification report
print(classification_report(y_true, y_pred))

# Visualize confusion matrix
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.savefig('confusion_matrix.png', dpi=300, bbox_inches='tight')

return results

```

1.5 3. Efficiency Metrics

1.5.1 3.1 Detection Latency

Definition: Time from packet arrival to detection decision

Measurement:

```

import time

def measure_latency(model, X_test, num_iterations=1000):
    """
    Measure detection latency
    """
    latencies = []

    for i in range(num_iterations):
        start_time = time.time()
        prediction = model.predict(X_test[i:i+1])
        end_time = time.time()

        latency = (end_time - start_time) * 1000 # Convert to ms
        latencies.append(latency)

    results = {
        'mean_latency_ms': np.mean(latencies),
        'median_latency_ms': np.median(latencies),
        'p95_latency_ms': np.percentile(latencies, 95),
    }

```

```

        'p99_latency_ms': np.percentile(latencies, 99),
        'max_latency_ms': np.max(latencies)
    }

```

```

    return results

```

Targets: - Mean latency: **<100ms** (general scenarios) - P99 latency: **<200ms** - Critical systems: **<50ms**

1.5.2 3.2 Throughput

Definition: Number of packets/flows processed per second

Measurement:

```

def measure_throughput(model, X_test, duration=60):
    """
    Measure detection throughput
    """
    start_time = time.time()
    processed_count = 0

    while time.time() - start_time < duration:
        model.predict(X_test[:1000]) # Batch of 1000
        processed_count += 1000

    throughput = processed_count / duration

    print(f"Throughput: {throughput:.0f} packets/second")
    return throughput

```

Targets: - Minimum: **>10,000 packets/second** - High-speed networks: **>100,000 packets/second**

1.5.3 3.3 Computational Resource Usage

CPU Utilization:

```

import psutil
import os

def measure_cpu_usage(model, X_test, duration=60):
    """
    Measure CPU usage during detection
    """
    process = psutil.Process(os.getpid())

    cpu_percentages = []
    start_time = time.time()

```

```

while time.time() - start_time < duration:
    cpu_percent = process.cpu_percent(interval=1)
    cpu_percentages.append(cpu_percent)

    # Perform detection
    model.predict(X_test[:100])

avg_cpu = np.mean(cpu_percentages)
max_cpu = np.max(cpu_percentages)

return {'avg_cpu_%': avg_cpu, 'max_cpu_%': max_cpu}

```

Memory Usage:

```

def measure_memory_usage(model):
    """
    Measure memory footprint
    """
    process = psutil.Process(os.getpid())
    memory_info = process.memory_info()

    memory_mb = memory_info.rss / 1024 / 1024 # Convert to MB

    # Model size
    if hasattr(model, 'count_params'):
        params = model.count_params()
        model_size_mb = params * 4 / 1024 / 1024 # Assuming float32
    else:
        model_size_mb = None

    return {
        'process_memory_mb': memory_mb,
        'model_size_mb': model_size_mb
    }

```

Targets: - CPU usage: <50% (single core) - Memory: <2GB (edge devices), <8GB (servers) - Model size: <100MB (edge), <500MB (cloud)

1.5.4 3.4 Energy Consumption (for edge devices)

```

def estimate_energy_consumption(power_watts, duration_hours):
    """
    Estimate energy consumption
    """
    energy_wh = power_watts * duration_hours
    cost_per_kwh = 0.12 # USD
    cost = energy_wh / 1000 * cost_per_kwh

    return {

```



```

        'energy_wh': energy_wh,
        'estimated_cost_usd': cost
    }

```

Target: Minimal power consumption for battery-operated edge devices

1.6 4. Scalability Metrics

1.6.1 4.1 Performance vs. Network Size

Measurement:

```

def evaluate_scalability(model, base_size=100):
    """
    Evaluate performance across different network sizes
    """
    network_sizes = [100, 500, 1000, 5000, 10000, 50000, 100000]
    results = []

    for size in network_sizes:
        # Generate synthetic data for 'size' devices
        X_scaled = generate_traffic(num_devices=size)

        # Measure metrics
        start_time = time.time()
        predictions = model.predict(X_scaled)
        detection_time = time.time() - start_time

        throughput = len(X_scaled) / detection_time
        latency = detection_time / len(X_scaled) * 1000 # ms

        results.append({
            'network_size': size,
            'throughput_pps': throughput,
            'latency_ms': latency
        })

    df = pd.DataFrame(results)

    # Plot
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    ax1.plot(df['network_size'], df['throughput_pps'], marker='o')
    ax1.set_xlabel('Network Size (devices)')
    ax1.set_ylabel('Throughput (packets/s)')
    ax1.set_title('Throughput vs. Network Size')
    ax1.set_xscale('log')

```

```

ax2.plot(df['network_size'], df['latency_ms'], marker='o', color='red')
ax2.set_xlabel('Network Size (devices)')
ax2.set_ylabel('Latency (ms)')
ax2.set_title('Latency vs. Network Size')
ax2.set_xscale('log')

plt.tight_layout()
plt.savefig('scalability_analysis.png', dpi=300)

return df

```

Target: Near-linear scaling (latency increase $<2\times$ when network size increases $10\times$)

1.6.2 4.2 Distributed System Performance

Speedup:

$\text{Speedup} = T_{\text{sequential}} / T_{\text{parallel}}$

Efficiency:

$\text{Efficiency} = \text{Speedup} / \text{Number_of_Nodes}$

Target: Efficiency $>80\%$ with distributed deployment

1.7 5. Adaptability Metrics

1.7.1 5.1 Zero-Day Attack Detection Rate

Definition: Ability to detect previously unseen attack types

Measurement:

```

def evaluate_zero_day_detection(model, X_known, y_known, X_unknown, y_unknown):
    """
    Evaluate zero-day detection capability
    """
    # Train on known attacks
    model.fit(X_known, y_known)

    # Test on unknown (zero-day) attacks
    y_pred_unknown = model.predict(X_unknown)

    # Binary classification: Normal vs. Anomaly
    y_unknown_binary = (y_unknown != 'Normal').astype(int)
    y_pred_binary = (y_pred_unknown != 'Normal').astype(int)

    zero_day_detection_rate = recall_score(y_unknown_binary, y_pred_binary)
    zero_day_fpr = false_positive_rate(y_unknown_binary, y_pred_binary)

```

```

results = {
    'zero_day_detection_rate': zero_day_detection_rate,
    'zero_day_fpr': zero_day_fpr
}

```

```

return results

```

Target: >95% detection rate for zero-day attacks

1.7.2 5.2 Concept Drift Handling

Measurement:

```

from river import drift

```

```

def evaluate_concept_drift_handling(model, data_stream):

```

```

    """
    Evaluate adaptation to concept drift
    """

```

```

    drift_detector = drift.ADWIN()

```

```

    accuracies = []
    drift_points = []

```

```

    for i, (x, y) in enumerate(data_stream):

```

```

        # Predict
        y_pred = model.predict([x])[0]
        correct = int(y_pred == y)

```

```

        # Update drift detector
        drift_detector.update(correct)

```

```

        if drift_detector.drift_detected:
            drift_points.append(i)
            print(f"Concept drift detected at sample {i}")
            # Model should adapt here (online learning)

```

```

        # Track accuracy over time
        accuracies.append(correct)

```

```

    # Performance degradation over time

```

```

    window_size = 1000
    windowed_accuracies = [
        np.mean(accuracies[i:i+window_size])
        for i in range(0, len(accuracies) - window_size, window_size)
    ]

```

```

    avg_degradation = (windowed_accuracies[0] - windowed_accuracies[-1]) / windowed_acc

```

```

results = {
    'drift_points': drift_points,
    'num_drifts': len(drift_points),
    'accuracy_degradation': avg_degradation
}

return results

```

Target: Accuracy degradation <5% over 6-month period

1.7.3 5.3 Adaptation Time

Definition: Time to adapt to new attack patterns

Measurement:

```

def measure_adaptation_time(model, X_new_attack, y_new_attack):
    """
    Measure time to adapt to new attack type
    """
    start_time = time.time()

    # Incremental learning
    model.partial_fit(X_new_attack, y_new_attack)

    adaptation_time = time.time() - start_time

    # Test if adapted
    accuracy = model.score(X_new_attack, y_new_attack)

    return {
        'adaptation_time_seconds': adaptation_time,
        'post_adaptation_accuracy': accuracy
    }

```

Target: Adaptation in <1 hour with online learning

1.8 6. Explainability Metrics

1.8.1 6.1 Feature Importance Consistency

Measurement:

```

def evaluate_feature_importance_consistency(model, X_test, num_runs=10):
    """
    Evaluate consistency of feature importance across runs
    """
    feature_importances = []

```

```

for _ in range(num_runs):
    # Compute SHAP values
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_test)

    # Global feature importance
    importance = np.abs(shap_values).mean(axis=0)
    feature_importances.append(importance)

feature_importances = np.array(feature_importances)

# Compute consistency (low std = high consistency)
consistency = 1 - (np.std(feature_importances, axis=0) / np.mean(feature_importances))
avg_consistency = np.mean(consistency)

return {'avg_consistency': avg_consistency, 'per_feature_consistency': consistency}

```

Target: Average consistency >0.90

1.8.2 6.2 Explanation Fidelity

Definition: How faithfully explanations represent model behavior

Measurement:

```

def evaluate_explanation_fidelity(model, X_test, lime_explainer):
    """
    Evaluate LIME explanation fidelity
    """
    fidelities = []

    for i in range(100):
        x = X_test[i]

        # Original prediction
        orig_pred = model.predict_proba([x])[0]

        # LIME explanation
        explanation = lime_explainer.explain_instance(x, model.predict_proba)

        # Local surrogate prediction
        surrogate_pred = explanation.local_pred[0]

        # Fidelity = similarity between original and surrogate
        fidelity = 1 - abs(orig_pred[1] - surrogate_pred)
        fidelities.append(fidelity)

    avg_fidelity = np.mean(fidelities)

```

```
return {'avg_fidelity': avg_fidelity}
```

Target: Average fidelity >0.85

1.8.3 6.3 User Trust Score (Human Evaluation)

Methodology: - Recruit 10-20 security analysts - Present alerts with and without explanations - Measure trust, confidence, understanding

Questionnaire (5-point Likert scale): 1. I understand why this alert was generated
2. I trust the IDS's decision 3. The explanation helps me take appropriate action 4. The key features highlighted make sense

Target: Average score >4.0/5.0

1.9 7. Privacy Metrics

1.9.1 7.1 Differential Privacy Guarantee

Definition: Privacy loss parameter ϵ

$$\Pr[M(D) \in S] \leq e^{\epsilon} \times \Pr[M(D') \in S] + \delta$$

where: - M: privacy mechanism - D, D': neighboring datasets differing in one record
- ϵ : privacy budget - δ : failure probability

Target: $\epsilon < 1.0$, $\delta < 10^{-5}$

1.9.2 7.2 Information Leakage

Measurement:

```
def measure_information_leakage(model, X_train, X_test):  
    """  
    Measure if model memorizes training data  
    """  
    # Membership inference attack  
    train_losses = []  
    test_losses = []  
  
    for x_train in X_train[:1000]:  
        loss = compute_loss(model, x_train)  
        train_losses.append(loss)  
  
    for x_test in X_test[:1000]:  
        loss = compute_loss(model, x_test)  
        test_losses.append(loss)  
  
    # Lower loss on training data indicates memorization
```

```

avg_train_loss = np.mean(train_losses)
avg_test_loss = np.mean(test_losses)

leakage_score = (avg_test_loss - avg_train_loss) / avg_test_loss

return {'leakage_score': leakage_score}

```

Target: Leakage score <0.1 (minimal difference)

1.10 8. Comparative Evaluation Framework

1.10.1 8.1 Baseline Comparisons

Baselines: 1. Traditional ML: Random Forest, SVM, Decision Tree 2. Deep Learning: CNN, LSTM, GRU (standalone) 3. Existing IDS: Snort, Suricata, Zeek 4. Recent Research: State-of-the-art models from recent papers

1.10.2 8.2 Comparative Table Template

Model	Accuracy	Precision	Recall	F1	AUC-ROC	FPR	Latency (ms)	Throughput (pps)
Proposed	-	-	-	-	-	-	-	-
CNN-LSTM-Transformer	-	-	-	-	-	-	-	-
Random For-est	-	-	-	-	-	-	-	-
SVM	-	-	-	-	-	-	-	-
Standaløne CNN	-	-	-	-	-	-	-	-
Standaløne LSTM	-	-	-	-	-	-	-	-
Snort	-	-	-	-	-	-	-	-
Recent Pa-per [X]	-	-	-	-	-	-	-	-

1.11 9. Statistical Significance Testing

1.11.1 9.1 Cross-Validation

k-Fold Cross-Validation:

```

from sklearn.model_selection import cross_val_score

def cross_validate_model(model, X, y, k=10):
    """
    Perform k-fold cross-validation
    """
    scores = cross_val_score(model, X, y, cv=k, scoring='accuracy')

    results = {
        'mean_accuracy': np.mean(scores),
        'std_accuracy': np.std(scores),
        '95%_confidence_interval': (
            np.mean(scores) - 1.96 * np.std(scores) / np.sqrt(k),
            np.mean(scores) + 1.96 * np.std(scores) / np.sqrt(k)
        )
    }

    return results

```

1.11.2 9.2 Statistical Tests

Paired t-test:

```

from scipy.stats import ttest_rel

def compare_models_ttest(model1_scores, model2_scores):
    """
    Compare two models using paired t-test
    """
    t_stat, p_value = ttest_rel(model1_scores, model2_scores)

    alpha = 0.05
    if p_value < alpha:
        result = "Statistically significant difference"
    else:
        result = "No significant difference"

    return {
        't_statistic': t_stat,
        'p_value': p_value,
        'result': result
    }

```

McNemar's Test (for binary classification):

```

from statsmodels.stats.contingency_tables import mcnemar

def mcnemar_test(model1_preds, model2_preds, y_true):
    """

```



```

McNemar's test for comparing classifiers
"""
# Contingency table
both_correct = np.sum((modell_preds == y_true) & (model2_preds == y_true))
both_wrong = np.sum((modell_preds != y_true) & (model2_preds != y_true))
modell_correct_model2_wrong = np.sum((modell_preds == y_true) & (model2_preds != y_
modell_wrong_model2_correct = np.sum((modell_preds != y_true) & (model2_preds == y_

table = [[both_correct, modell_correct_model2_wrong],
          [modell_wrong_model2_correct, both_wrong]]

result = mcnemar(table, exact=True)

return {
    'statistic': result.statistic,
    'p_value': result.pvalue
}

```

1.12 Conclusion

This comprehensive evaluation framework ensures rigorous assessment of the proposed AI-IDS across all critical dimensions: detection performance, efficiency, scalability, adaptability, explainability, and privacy. The combination of quantitative metrics, statistical testing, and human evaluation provides confidence in the system's real-world viability.

Key Evaluation Principles: - Multi-faceted assessment (not just accuracy) - Comparison against strong baselines - Statistical significance testing - Real-world performance simulation - Human-in-the-loop evaluation for explainability

Success Criteria Summary: - Detection: Accuracy >98%, F1 >97%, FPR <1% - Efficiency: Latency <100ms, Throughput >10K pps - Scalability: Linear scaling to 100K devices - Adaptability: Zero-day detection >95% - Explainability: User trust score >4.0/5.0 - Privacy: $\epsilon < 1.0$ in federated scenarios

Next: Refer to ../timeline/gantt_chart.md for research timeline and ../resources/budget.md for resource allocation details.